

PRELIMINARY PERFORMANCE ANALYSIS OF THE NEMO/OPA CODE

Sergiy Khan (skhan at ap.smu.ca), ACEnet, Saint Mary's University, Halifax

Original version: July 4, 2008; additions: July 24, 2008; the final version: Aug 25, 2008.

Introduction

The main objective of this study was to benchmark the Nemo code, analyse performance differences, and determine which machines at ACEnet are suited best for running Nemo.

Taking into account that the Nemo ACEnet community is really looking forward to learn about the results, this brief report is being released in somewhat hurry fashion. At the same time the analysis has not revealed anything unusual or extraordinary about the performance of the code, and the preliminary report can be exactly what is expected by the community.

Results

I was running the standard EXP01 benchmark test. This is the quote from the benchmark instructions by Frederic Dupont: "The configuration consists of a grid for the North Atlantic (492x326x46 in size) and an embedded region for the Gulf Stream region (409x259x46 in size). The embedding is achieved using the AGRIF package, which sets up the model to operate on both the main grid and the embedded grid, and handles the exchange of data between them."

The code was compiled with the default ACEnet PGI compiler (v 7.1 64-bit) and the Open MPI library, without optimization.

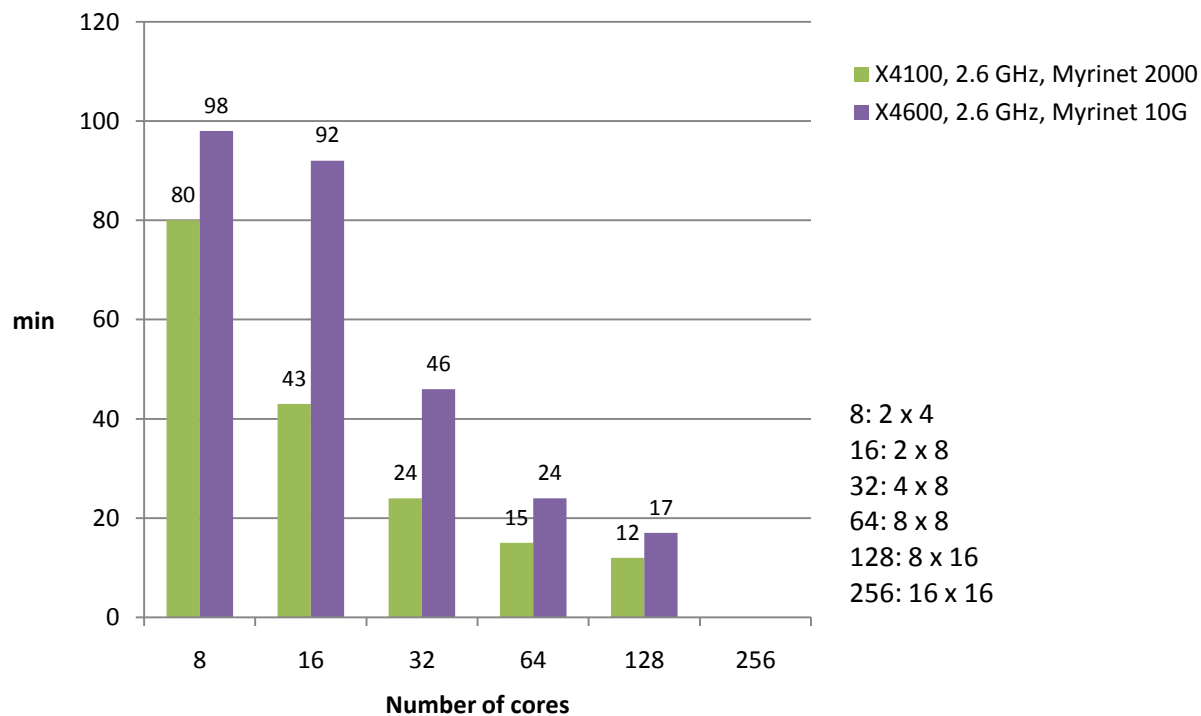
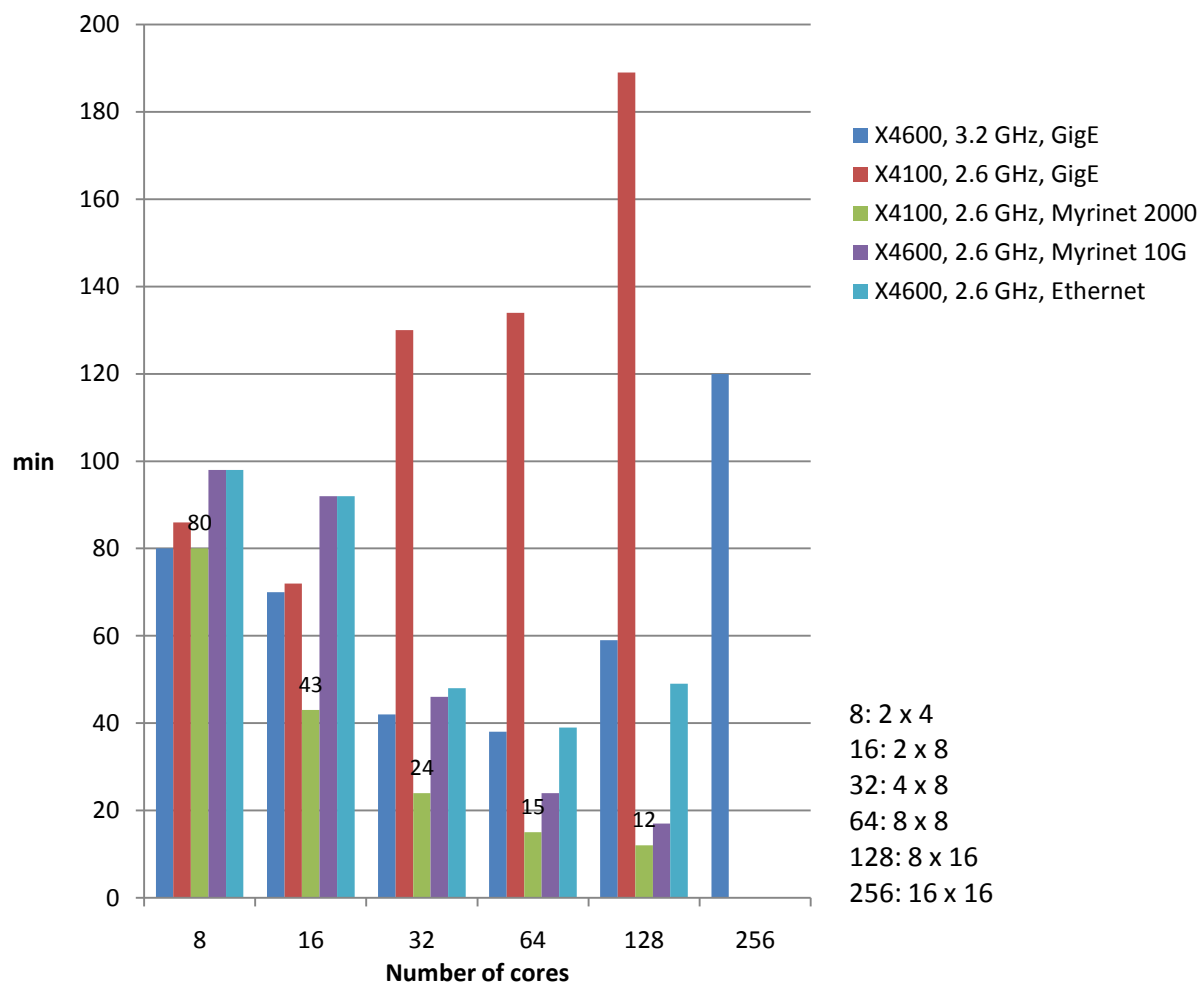
The tests were performed on the Linux clusters: Fundy, Placentia and Mahone, which represent the two main types of computational resources found at ACEnet: Sun Fire X4600 (16-core) and Sun Fire X4100/2200 (4-core), which are of essentially different architectures. Please note that depending on the time when these machines were acquired, they operate at different frequencies. The nodes on which my jobs ran operate at 2.6 GHz (X4100), and at 3.2 GHz and 2.6 GHz (X4600). Also, the Mahone cluster is fully equipped with a fast Myrinet interconnection (Myrinet 2000), while only twelve nodes at Placentia have Myrinet 10G cards.

I used the same executable files on both clusters for all tests. No recompilation was required since both clusters are running the same OS – Linux, and the Open MPI library allows using any available interconnection without recompiling the code (the fastest one by default).

The figures below represent the results as a bar chart: the run time in minutes versus a size of the job, i.e. a number of cores (processes). In the right part of the diagram, there is also a notation of how exactly the domain decomposition was configured in `par_oce.F90`.

Please note that there is a number of uncertainties contributing to the resulting error bar associated with every measurement. Unfortunately, I cannot reliably estimate them, although I make some comments about this below.

The data for the nodes equipped with the Myricom cards are also represented in a separate chart for better visual comparison.



Conclusions / Analysis

There is nothing unusual about these results; the performance dependencies are quite expected. Nonetheless, these are some basic conclusions I have come to looking at the graphs.

1. The performance of both the X4100 (2.6 GHz) and X4600 (3.2 GHz) is comparable for jobs with a number of processes up to 16, if no fast interconnection is used.
2. The performance increases (run time decreases) with the growing number of processes (cores). However this pattern breaks beyond a certain number and turns the opposite if the job is running over Gigabit Ethernet (GigE), i.e. the performance is getting worse as the number of processes increases. For the X4100, this boundary is at 16 cores, and for the X4600, it's 64. Please note that in both cases the number of processes corresponds to four nodes.
3. Jobs running over Myrinet demonstrate the best results and scalability. I believe that a larger simulation is in order to get proper measurements for the 64+ jobs.
4. I also note that for 16+ process jobs running over Ethernet, the variations in run time can be substantial, and depends a lot on the Ethernet network architecture and switching equipment, as well as on the overall network traffic.
On the other hand, jobs running over Myrinet show very consistent execution times.
5. The performance difference between jobs running over Myrinet on X4100s (2.6 GHz) and X4600s (2.6 GHz) diminishes with the growing number of processes.

Which machines to run on?

1. Small jobs (8- and 16-core) can be run on any cluster.
Since an 8-core job on the X4600 will likely to share the resources with other jobs, degrading the overall performance, I recommend running a 16-core job instead, taking all the slots (cores) on the node using the openmp parallel environment.
Please keep in mind that this advice is not applicable to 4-way nodes due to a different architecture.
2. Large jobs (16+) will run much better on the nodes equipped with a fast interconnection.
3. Do not run jobs over Ethernet if they span more than 4 nodes. The highest acceptable number of processes is 64 for the X4600s, and it's 16 for the X4100s. You will not get a better performance; moreover the performance can worsen substantially depending on the current Ethernet network traffic.

Useful Notes

Myrinet

The Mahone cluster is fully equipped with Myricom cards. Twelve nodes on Placentia (ex-Solaris nodes) are also equipped with Myrinet cards, and will soon be available to general users.

netCDF

I also advise everyone that there is a new version (3.6.3) of the netCDF library installed on every cluster in `/usr/local/netCDF3`.

Makefile

The ACEnet version of Makefile for the NEMO code is available on request.

Node depopulation

The presentation about the Nemo Benchmark Code was referred to several times during the initial meeting with the NEMO community. Although later on it was indicated to me that this presentation was not meant to be compared to, I still feel that for many users it serves as a reference point, thus I would like to make some comments on the controversial subject of node depopulation.

Several slides of the presentation contain the following statement: “Major improvement in Xeon performance with decreasing ppn.” And the conclusion is “At a given process count, the Xeon performance is doubled when depopulating the codes e.g. ppn=2.” I would like to emphasize that they are referring to the Xeon CPU performance, not to the performance or scalability of the code.

Now let’s think for a second about the aforementioned conclusion. If you look at slide 40 (p. 26) or slide 24 (p. 10) then you can see indeed that for the case of ppn=4, it takes about twice as much time to run the benchmark when compared to the case of ppn=2. Does it mean that the Xeon performance has doubled for ppn=2? Please note, that in order to maintain the same number of processes, one will have to use twice as many CPUs. Is it surprising to get twice the performance then? In the end, we have got twice as many CPUs and twice the performance meaning that the performance of a single CPU stays the same. The only thing that can be remotely interpreted as “doubling its performance” is probably the per-core performance within one CPU.

Another possible misinterpretation is that for a given number of CPUs (or cores), one can improve the performance of the code by depopulating the nodes. As I look at the graphs in the presentation I see that the node depopulation does not make the code run faster. In fact, the effect is exactly the opposite – it takes longer to run the test! There is only one exception to this, which is for 4-way nodes running 128- or 256-process jobs in the configuration of ppn=2 vs ppn=4, they demonstrate about the same performance, that probably can be attributed to scalability problems or I/O performance issues. For all other cases, the performance is getting worse when depopulating the nodes.

Questions?

If you have questions about this report you can email me. Or we could arrange a meeting in person with NEMO users from Halifax, or via IOCOM with users from Newfoundland, or with everyone at once.

Useful Links

- ACEnet User Guide: http://wiki.ace-net.ca/index.php/User_Guide
- Open MPI library at ACEnet: http://wiki.ace-net.ca/index.php/Open_MPI

Acknowledgements

I would like to thank Adam Drozdowski and Frederic Dupont for providing me with the very clear and easy-to-follow set of instructions on how to compile the code and run the benchmark. I am also grateful to Ross Dickson for his feedback on this report.